Speaker notes

Mini project part 1 - unit testing the hotel booking core · target 14 minutes, hard limit 15

Team presentation: every team member must speak. Suggested split - slides 1-5 (introduction, design, mocking), slides 6-9 (data-driven tests and bugs), slides 10-12 (coverage, SonarCloud, conclusions). Change it to fit your team. Do **not** explain the CI pipeline itself; only how SonarCloud and the dashboard were used.

SLIDES	TOPIC	TIME
1-3	Introduction, plan, system under test	2:00
4-5	Test design and mocking	3:30
6-7	Data-driven tests	3:00
8-9	Bugs found	2:45
10-12	Coverage, SonarCloud, conclusions	3:00

1. Title

0:30

- Introduce the team and the project: a hotel booking system, and we tested its core business logic in `HotelBooking.Core`.
- Say what is coming: test design, mocking, a data-driven test, the bugs we found, SonarCloud.

2. Overview

0:15

- Read the eight points quickly. Do not explain them yet.

3. System under test

1:15

- All business rules are in `BookingManager`. It depends on two interfaces, `IRepository<Booking>` and `IRepository<Room>`.
- They are injected through the constructor. **That is what makes the class testable**: in tests we hand it fakes instead of a database.
- Three methods: `FindAvailableRoom` (start date in the future, no overlap with active bookings, else -1), `CreateBooking` (assign room, set active, save), `GetFullyOccupiedDates` (dates when all rooms are booked).

4. Test design 1:45

- 32 tests, xUnit v3 and Moq. Every test follows Arrange - Act - Assert.
- Names are sentences: `Method_Scenario_ExpectedResult`, e.g. `CreateBooking_NoRoomAvailable_ReturnsFalse`. A failing test name tells you what broke.
- The constructor builds the mocks and a default hotel with two rooms. Each test sets up only what it needs, with the helper `SetUpBookings`.
- Dates are relative to `DateTime.Today`, so the tests do not expire.
- Choosing cases: **equivalence classes** (invalid dates, no bookings, free room, all rooms taken, cancelled booking) and **boundary values** (day before, same day, day after a booking).

5. Mocking with Moq 1:45

- Why: the tests must check the logic, not the database. A mock is fast and fully under our control.
- **Stubbing:** `Setup(...).ReturnsAsync(...)` decides which rooms and bookings the manager sees.
- **Verification:** `Verify(r => r.AddAsync(booking), Times.Once)` proves the booking is saved exactly once when a room is free. `Times.Never` proves nothing is saved when the hotel is full.
- Point: verification checks *behaviour* (what the code did), not only its return value.

6. Data-driven test 2:15

- One test method, six cases, supplied by `[MemberData]`. Adding a case is one new line, not a new method.
- Setup: room 1 is booked from day 10 to day 20. Each row is *start offset, end offset, is room 1 available*.
- Walk through the timeline on the right. Two cases to point out: `{1, 10, false}` ends on the day the booking starts, so it overlaps. `{21, 25, true}` starts the day after it ends, so it is free.
- Why `MemberData` and not `InlineData`: `InlineData` needs compile-time constants, so dates cannot go in it. We pass day offsets and build the dates in the test.

7. Second theory 0:45

- Same idea for `GetFullyOccupiedDates`, but the data contains whole arrays of bookings and the expected dates.
- Three cases: both rooms booked for the full period, both booked for the middle day only, one booking inactive.

8. Bug 1 - bookings counted instead of rooms

1:45

- The method compared the number of **bookings** on a date with the number of rooms.
- Example: two rooms, two active bookings on **room 1**. The code counts 2 ≥ 2 and says fully occupied, but room 2 is free.
- We wrote the regression test first: it **failed** on the old code (red). Then we changed the condition to count **distinct room ids**, and it passed (green).
- The old line is kept as a comment marked OLD CODE (BUG), the new one marked FIXED. After the fix all 32 tests pass.

9. Bug 2 - off-by-one

1:00

- In `FindAvailableRoom` a comparison used `<=` where it needed `<`.
- The boundary case `{1, 10, false}` catches it: a stay ending on the day another booking starts was wrongly seen as free.
- Message: a typical example would never hit this edge. That is the value of boundary values in a data-driven test.

10. Coverage

0:45

- `BookingManager` has 100% line and branch coverage: measured locally with `dotnet-coverage`, and confirmed on the SonarCloud Code tab (`BookingManager.cs`, 0 uncovered lines).
- Caveat to say out loud: coverage is a floor, not proof. Both bugs were found by choosing the right inputs, not by the coverage number.

11. SonarCloud and Project health dashboard 1:30

- Open the live dashboard (SonarCloud Overview) or point at the screenshot. Four tiles: quality gate, open issues, duplications, coverage.
- **Quality gate: Passed**. Duplication 1.8%.
- **9 open issues:** 4 reliability issues, all medium impact, so the Reliability rating is **C** (C means at least one medium-impact issue). 5 maintainability issues, and the Maintainability rating is **A** (technical debt ratio below 5%). All 4 are in the starter code, none in `HotelBooking.Core` or the tests: `app.Run()` should be `RunAsync()` in the MVC and Web API `Program.cs` (async-await rule), `RenderSection` should be `RenderSectionAsync` and the `<html>` tag lacks a `lang` attribute in `_Layout.cshtml` (accessibility).
- **Coverage 21.4%** is for the whole solution. It also counts MVC, Web API and Infrastructure, which are outside the scope of the Core unit tests. on the SonarCloud Code tab, `BookingManager.cs` itself is listed at 100% with 0 uncovered lines - confirmed in SonarCloud, not just locally. Check the Code tab in SonarCloud and say what it shows for Core.
- Add: what we did with the dashboard, e.g. looked at the 9 issues, checked that coverage did not drop.
- Do not explain the CI pipeline itself.

12. Conclusions 0:45

- Dependency injection makes the code testable; mocks give speed and control; data-driven tests turn boundary analysis into a table.
- Tests found two real bugs. Write the failing test first, then fix, and it stays as a regression guard.
- Static analysis and coverage tell us where to look; well-chosen inputs tell us if the code is right.
- Finish: "Thank you, we are happy to answer questions."

Likely questions

Q: Mock or stub - what is the difference?
A stub only returns prepared data (`Setup/ReturnsAsync`). A mock is also checked for how it was called (`Verify`). Moq objects can do both.

Q: Why a Theory instead of several Facts?
Same logic with different data. One method is easier to maintain, and a new case is one line.

Q: Why are the dates relative to today?
`FindAvailableRoom` rejects dates that are not in the future. Fixed dates would start failing once they pass.

Q: Is 100% coverage enough?
No. It shows lines were executed, not that the assertions are right. That is why we chose boundary values and found two bugs.

Q: Why is SonarCloud coverage only 21.4%?
It counts the whole solution. The Core business logic is 100% locally; MVC and Infrastructure are not covered by these unit tests.

Q: How do you know the tests are good?
They failed on the buggy code (red) and pass on the fixed code (green). We can also break the code on purpose and check that a test fails.

Q: Why is the Reliability rating C?
SonarCloud gives C when there is at least one medium-impact reliability issue; we have 4. All 4 are in the MVC and Web API startup and layout files from the starter project (`Program.cs` x2, `_Layout.cshtml` x2), unchanged since the initial commit. None are in the business logic or the tests. They are quick fixes (2-5 minutes each) that we did not need for the unit-testing goal.